Project Report

Intro and Research Objective

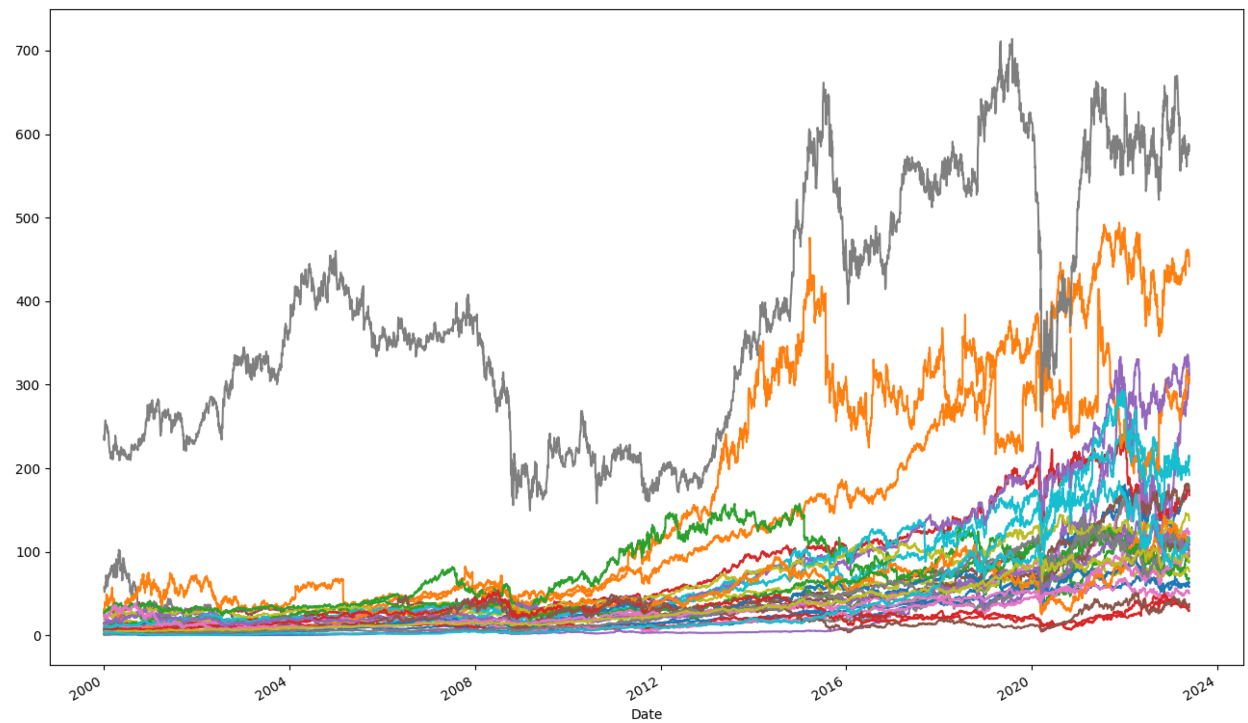
In finance, estimating stock returns is a challenging problem. Models that rely solely on historical returns when predicting future returns usually have low predictive performance. We thought it might be due to the fact the return series are stationary and memoryless, meaning that statistical properties of the series such as mean and standard deviation are constant, and there is no pattern that can be leveraged. Our hypothesis was that if we could transform the stock price data so that it becomes stationary, which would help us model the process more easily, and at the same time it preserves some of the memory of the stock prices, that would help us to better predict returns, which in turn would help to construct a better-performing portfolio.

To perform such a transformation, we found out a method called fractional differencing, which was invented a long time ago (1970) for exactly that purpose. To take advantage of preserved memory, we decided to use LSTM, which is a deep learning model specifically designed to capture long-term dependencies in the time series data. So, our final objective was to investigate whether applying fractional differencing on log stock prices enhances the accuracy of predicting future stock returns using LSTM and helps us build a better performing portfolio compared to a benchmark (portfolio build on the predictions coming from the historical returns).

Dataset

The dataset that we chose consists of adjusted close prices and volume data spanning from 2000 to 2023 for companies included in S&P 500 (since training period for our forecasting model was decided to be 10 years, we decide to look at the components of S&P 500 in 2010). As some companies had their stock tickers changed after 2010, we were not able to get the data for all 500 companies and only for about 400 companies. After removing companies that did not have a 10-

year trading history before 2010, we left with 300 companies. Out of those 300 companies, we randomly sampled 30 companies because it would have taken too much time (one or two days) to build a model for all companies even if we used GPUs. Here is the graph showing how those companies' stock prices evolved through time:



Model Building

Since we saw that there is a relationship between returns and fractionally differentiated log prices, we decided to directly forecast daily returns using fractionally differentiated log prices as a feature. Additionally, we included daily volume information as another feature to forecast returns. An LSTM model that is constructed using those features and returns (as predicting variable) will be used to build our target portfolio. We will also construct an LSTM model using historical returns and daily volume data as predictors for future returns. This is done for a benchmark model. We will conduct two experiments in which we will change the hyperparameters of the LSTM model and compare the results:

Experiment 1: 7 hidden nodes, 21 lag terms, 100 epochs and 128 batch size.

Experiment 2: 2 hidden nodes, 4 lag terms, 100 epochs and 128 batch size.

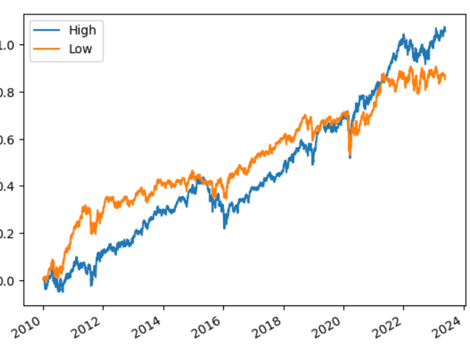
The second experiment is conducted because in the first experiment, we have a large number of lags (21), which might include unnecessary information into the model and thus decrease the performance of the model.

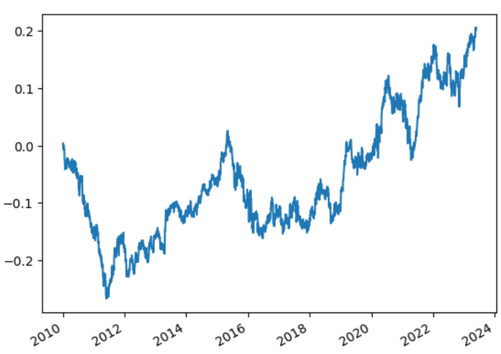
Portfolio Construction

After building an LSTM model on 10-year training data for each stock, we get its daily stock return prediction for the following year, and then each we day rank the chosen stocks based on the estimated stock returns. After that, we buy top 5 stocks ('High')) that had highest predicted returns and short 5 stocks ('Low') that had lowest predicted returns.

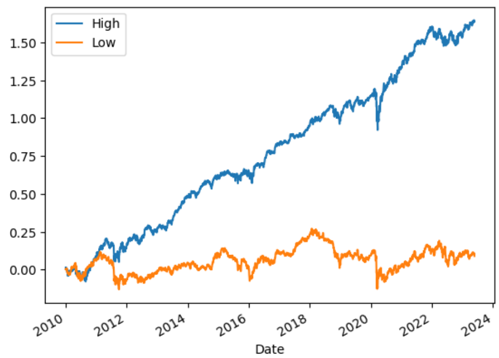
Experiment 1 Results

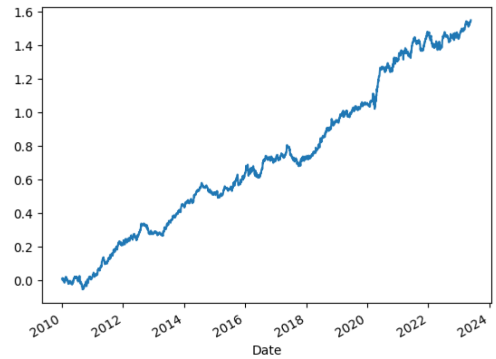
Target Portfolio Performance (y axis shows the cumulative returns):





Benchmark Portfolio Performance

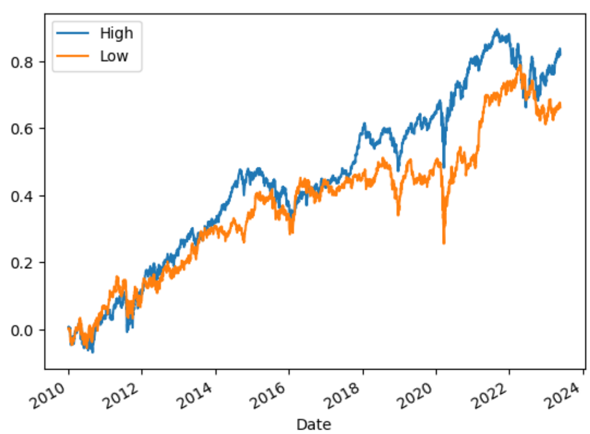


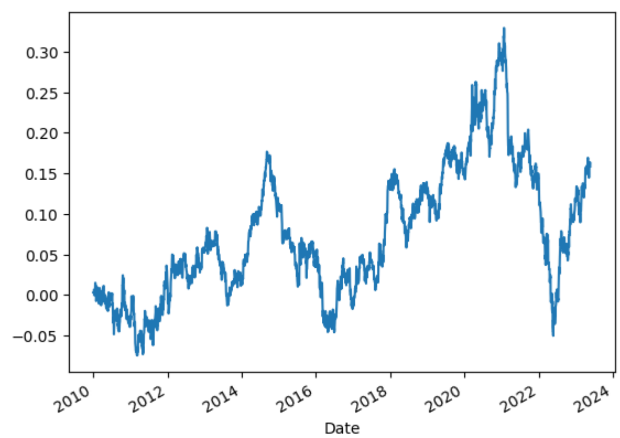


We can see that the benchmark portfolio performed considerably better than the target portfolio in the 1st experiment.

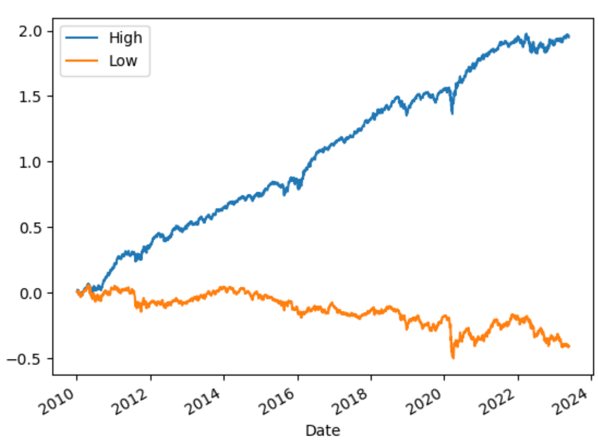
Experiment 2 Results

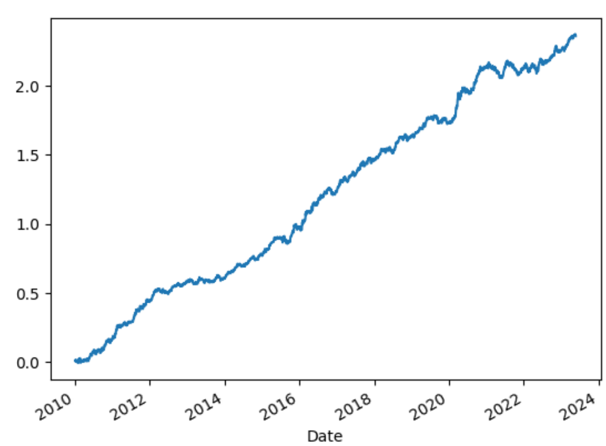
Target Portfolio Performance (y axis shows the cumulative returns):





Benchmark Portfolio Performance:





In the 2nd experiment, the target portfolio has a better performance than what we saw in the 1st experiment, but it is still outperformed by the benchmark portfolio.

Conclusion

In both experiments, the benchmark portfolio outperformed the target portfolio. This might be due to a variety of reasons. One reason can be that our chosen stock universe was too small. Also, we might have mis-specified and undertrained our LSTM model for the target portfolio. Another possible reason can be that fractional differencing might not actually be effective to predict stock returns (past data, regardless of some preserved historic pattern, does not predict future behavior). However fractionally differencing might turn out to be useful for other kinds of financial and economic data. To get a conclusive answer regarding the effect of fractional differencing on stock return prediction, more extensive analysis might be necessary.

References

1. Gu S., Kelly B. & Xiu D. (2018, September). Empirical Asset Pricing via Machine learning https://academic.oup.com/rfs/article-pdf/33/5/2223/33209812/hhaa009.pdf
2. Shah I. (2020, February). Historical Stock Price Data in Python https://towardsdatascience.com/historical-stock-price-data-in-python-a0b6dc826836
3. Gui K. (2020, August). Feature Engineering: Fractionally differentiated features https://medium.com/swlh/fractionally-differentiated-features-9c1947ed2b55
4. Example of Fractional differencing applied to S&P 500 index https://colab.research.google.com/github/fracdiff/fracdiff/blob/main/examples/example_prado.ipynb#scrollTo=P9dnVAHftZtn
5. Investment Portfolio Optimization using LSTM https://github.com/georgemuriithi/investment-portfolio-optim/tree/main
6. Shah I & Gupta A. (2021, September) Long Short Equity Strategy https://blog.quantinsti.com/long-short-equity-strategy/
7. Lopez De Prado M. (2018, February) Advances in Financial Machine Learning